บันทึกการใช้ AI และแหล่งอ้างอิง

ผลงาน: JobGuard | บันทึกจากการพัฒนาและหลักฐานในแชต | 27 ตุลาคม 2569

1. เครื่องมือที่ใช้

เครื่องมือ	ใช้ในขั้นตอนใด
Codex ในแอป Codex	อ่านแผน สร้าง Next.js/TypeScript/CSS เตรียมข้อมูล เขียน/รันทดสอบ และร่างเอกสาร ไม่ระบุชื่อรุ่นย่อยที่ยืนยันไม่ได้
Image Gen	สร้างแนวทางหน้าจอ ก่อนทำหน้าจอจริงด้วยโค้ด ไม่ใช้ภาพแทน UI
In-app browser (IAB)	ตรวจ UI กรอก URL เลือกไฟล์จำลอง บันทึกการตัดสินใจ โหลดใหม่ และเก็บภาพหลักฐาน
Node test runner / Web Crypto / npm	ทดสอบตรรกะ เทียบ SHA-256 กับ Node crypto ติดตั้งและ build โปรเจกต์

2. ส่วนที่ AI ช่วย และบทบาทของผู้ใช้ที่ยืนยันได้

ขั้นตอน	AI ช่วยอะไร	สิ่งที่ยืนยันได้จากการทำงาน
กำหนดปัญหา/ขอบเขต	แปลงแผนเป็นโครงสร้างและรายการงาน	ผู้ใช้ให้แผน JobGuard กลุ่มเป้าหมาย แหล่งข้อมูล และเกณฑ์ส่งมอบ
ออกแบบและลงโค้ด	สร้างหน้าจอ 2 แท็บ checker และ history	AI เลือกรายละเอียด implementation ตามแผน ยังไม่มีบันทึกการตรวจโค้ดด้วยตนเองของผู้ใช้
เตรียมข้อมูล	ดึง URL/แฮชอย่างเดียว ยึด commit และแนบ license	ไม่มีการดาวน์โหลดไฟล์มัลแวร์ ข้อมูลทดสอบแยกจากข้อมูลภัยจริง
ทดสอบและแก้ไข	รัน 11 tests ตรวจ UI และเก็บผลจริง	ผู้ใช้รายงาน ERR_CONNECTION_REFUSED หลังเปิดเว็บ จึงเปิด server ใหม่และตรวจ HTTP 200
เอกสาร	เรียบเรียงสรุป ผลทดสอบ และ AI log	ผู้ใช้ให้ PDF ตัวอย่างและขอปรับเป็นเนื้อหาโปรเจกต์นี้

3. ตัวอย่างแผนและแนวทางที่นำไปใช้

สรุปประเด็นจากแผนพัฒนา JobGuard และผลที่นำไปทำจริง

ตัวอย่างที่ 1 - วางแผนต้นแบบให้จบกระบวนการ

แผนย่อ: ทำเว็บ 2 แท็บสำหรับตรวจ URL และไฟล์ ให้ผู้ใช้ใส่ข้อมูล อ่านผลพร้อมหลักฐาน เลือกการกระทำต่อ และกลับมาดูประวัติได้ โดยตรวจทั้งหมดในเบราว์เซอร์

ผลที่นำไปทำ: แยกส่วน URL checker, file checker และ history storage รองรับ URL สูงสุด 5 ลิงก์ ไฟล์ครั้งละ 1 ไฟล์ไม่เกิน 20 MB พร้อมบันทึกการตัดสินใจ

ตัวอย่างที่ 2 - ใช้แหล่งข้อมูลจาก GitHub

แนวทาง: ใช้รายการ URL ฟิชชิงจาก Phishing.Database และรายการ SHA-256 จาก ESET malware-ioc ที่เผยแพร่บน GitHub ตรวจเงื่อนไขใบอนุญาต และดาวน์โหลดเฉพาะรายการข้อมูล ไม่ดาวน์โหลดตัวอย่างมัลแวร์

ผลที่นำไปทำ: ตรวจรูปแบบ ตัดซ้ำ และจัดเก็บเป็น data/threats.json มี URL 5,000 รายการและแฮช 41 รายการ พร้อมแหล่งที่มา commit วันที่ดาวน์โหลด และใบอนุญาต ใช้เทียบในเครื่องโดยไม่เรียก GitHub ทุกครั้งที่ตรวจ

ตัวอย่างที่ 3 - อธิบายผลและรักษาความเป็นส่วนตัว

แผนย่อ: แยกผลการจับคู่ข้อมูลภัยออกจากการตรวจชนิดไฟล์ อธิบายข้อจำกัดให้ชัด และไม่เก็บไฟล์ URL เต็ม หรือข้อความแชทในประวัติ

ผลที่นำไปทำ: แสดงสถานะพร้อมหลักฐานและคำเตือน “ไม่พบรายการ ไม่ได้แปลว่าปลอดภัย” เก็บประวัติย่อใน localStorage สูงสุด 100 รายการ ใช้แฮชจำรายการซ้ำ และไม่อัปโหลดไฟล์ของผู้ใช้

การตรวจแก้ แหล่งข้อมูล และขอบเขตความรับผิดชอบ

JobGuard | ส่วนที่แก้จริงและสิ่งที่ยังต้องตรวจทานก่อนนำส่ง

4. จุดที่ปรับแก้ระหว่างพัฒนา

สิ่งที่พบ	ปรับแก้เป็น	เหตุผลและผลที่ยืนยันได้
ตัดเครื่องหมายท้าย URL อาจเปลี่ยน path /query	รักษาอักขระท้าย URL ไม่เดาเพื่อตัดทิ้ง	เพิ่ม assertion สำหรับ query ที่ลงท้ายด้วยจุด ทดสอบผ่าน
ล้างประวัติแต่ผลตรวจเดิมยังเลือกการตัดสินใจได้	ล้างผลที่แสดงพร้อมประวัติ	ป้องกันสถานะเหมือนบันทึกสำเร็จในรายการที่ไม่มีประวัติแล้ว ยังไม่ได้กดปุ่มล้างใน UI จริง
tsx CLI สร้าง IPC ไม่ได้ในสภาพแวดล้อมจำกัด	node --import tsx --test tests/*.test.ts	คำสั่งทดสอบรันได้จริงครบ 11 กรณี
ตัวอักษรไทยและระยะหน้าจออ่านยากในรอบแรก	รวม Noto Sans Thai ในโปรเจกต์ ปรับ CSS	ตรวจภาพเดสก์ท็อปและมือถือ ไม่มี CDN ฟอนต์ขณะตรวจ
เอกสารสรุป HTML เดิมหน้าสองทับเลขหน้า	ลดขนาดและระยะบรรทัดหน้าสอง	ตรวจตำแหน่งเนื้อหาและภาพเรนเดอร์แล้วไม่ทับ footer

วิธีตรวจสอบความถูกต้องที่ทำจริง

- รัน npm test, npm run typecheck และ npm run build ดูผลบันทึกใน docs/evidence/ ไม่อ้างว่าทำ mutation testing เพราะยังไม่ได้ทำ
- เทียบ SHA-256 ของไฟล์ที่สร้างเองกับ Node createHash และค่ามาตรฐานของ abc แยกจากทดสอบ matcher ด้วยแฮชจริงของ ESET
- ตรวจ IAB บันทึกการตัดสินใจแล้ว reload และดับเบิลคลิกไฟล์จำลอง 20 MiB ตรวจ Resource Timing ของ URL/ไฟล์ที่ทดสอบแล้วไม่มีคำขอใหม่หลังเริ่มตรวจ

5. แหล่งที่มาของข้อมูล โค้ด และส่วนประกอบ

รายการ	ที่มาและเงื่อนไข
ข้อมูล URL จริง	Phishing-Database/Phishing.Database, phishing-links-ACTIVE.txt, MIT; parse/normalize/unique/sort แล้วเลือก 5,000 URL แรก
ข้อมูล SHA-256 จริง	eset/malware-ioc, deceptivedevelopment/samples.sha256, BSD-2-Clause; ใช้ทั้ง 41 รายการจากงานวิจัยที่เลือก
Metadata และใบอนุญาต	data/threats.json ระบุไฟล์ต้นทาง commit เต็ม วันที่ และจำนวน; ใบอนุญาตใน data/licenses/ และ public/licenses/
โค้ดและ dependencies	โค้ดโปรเจกต์เขียนในงานนี้ด้วยความช่วยเหลือ AI ใช้ Next.js, React, TypeScript, lucide-react และเครื่องมือพัฒนาตาม package-lock.json
ฟอนต์และภาพ	Noto Sans Thai ผ่าน @fontsource, SIL OFL 1.1; ภาพแนวทางโดย Image Gen และภาพหลักฐานจาก IAB
ข้อมูลทดสอบและงบ	ไฟล์จำลองสร้างเอง ไม่มีมัลแวร์; งบ 30,000 บาทและเป้าหมาย 80% เป็นข้อเสนอจากแผน ยังไม่ใช่ผลหรือราคายืนยัน

แหล่งอ้างอิง: github.com/Phishing-Database/Phishing.Database และ github.com/eset/malware-ioc/tree/master/deceptivedevelopment รายละเอียด commit เต็มอยู่ใน data/threats.json

6. สิ่งที่ต้องยืนยันก่อนนำส่ง

ผลในเอกสารอ้างจากการรันและหลักฐานที่มี ไม่แต่งผลผู้ใช้จริง ไม่รับรองความปลอดภัยของ URL/ไฟล์ ผู้ใช้ควรอ่านโค้ดและใบอนุญาต ทดลองรันเอง และยืนยันว่าตนเข้าใจเหตุผลการออกแบบ ยังไม่มีหลักฐานในแชตว่าผู้ใช้ตรวจโค้ดทั้งหมดหรือดำเนินกิจกรรมกับผู้เข้าร่วมแล้ว